

universidad de León Online

Universidad de León ▪ Máster Universitario en Ciberseguridad

Ingeniería del software seguro en la era de la IA generativa

Diseño y Programación Seguras

Pipeline de validación automática

Francisco J. Rodríguez Lera

Material elaborado con asistencia de IA generativa (DeepSeek-V4-Flash; considerados también GPT-6 y Claude Sonnet 4.6). Contenido revisado y validado por el autor.



Instituto de Investigación e Innovación en Ingeniería

Escuela de Ingenierías Industrial, Informática y Aeroespacial

universidad de León

Objetivos de aprendizaje

Al terminar el capítulo, el estudiante será capaz de:

- **Distinguir** validar (comprobar propiedades declaradas) de demostrar seguridad.
- **Describir** la arquitectura del pipeline y sus etapas.
- **Comparar** técnicas de análisis y sus límites (falsos positivos y negativos).
- **Diseñar** puertas de calidad *fail-closed* y casos de rechazo.
- **Explicar** por qué un pipeline verde no equivale a ausencia de vulnerabilidades.

De la generación a la evidencia

- Un artefacto generado es una *hipótesis de solución*, no una solución verificada.
- El pipeline somete esa hipótesis a comprobación y *registra* el resultado de cada una.
- Su producto no es un veredicto binario, sino un *expediente*: qué se comprobó, con qué herramientas, sobre qué versión, con qué resultado y qué quedó sin comprobar.
- La IA traslada el esfuerzo hacia especificación, revisión y validación; el pipeline hace esa validación repetible a escala.

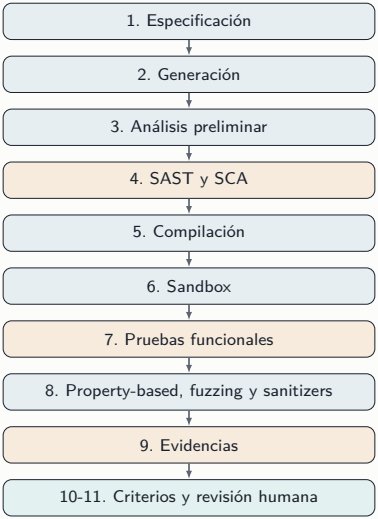
La automatización no sustituye al juicio: automatiza comprobaciones *expresables*; no decide si un requisito estaba bien formulado ni si un diseño es adecuado.

Etiquetado epistémico inicial

- **Propuesta docente:** el diseño concreto del pipeline (etapas, puertas, criterios).
- **Evidencia:** que las técnicas de análisis existen y tienen los límites descritos.
- **Hipótesis:** que una configuración concreta baste para *este* artefacto y *estas* condiciones.

En ningún caso se afirma que superar el pipeline equivalga a ausencia de vulnerabilidades.

Arquitectura del pipeline (orden de dependencia)



La figura es un orden de dependencia

- Cada bloque presupone el anterior y produce las condiciones del siguiente.
- No se ejecuta código en el sandbox antes de que la especificación declare qué se espera y qué límites se imponen.
- Que las etapas aparezcan encadenadas no significa que cada una «cubra» la anterior ni que su superación conjunta demuestre seguridad.
- Ejecutar *fuzzing* sin límites de tiempo en el sandbox convierte la prueba en un riesgo.

Las etapas naranjas son *puertas de calidad* que pueden rechazar el artefacto.

Etapas del pipeline (1-11)

- **1. Especificación:** requisitos funcionales y de seguridad (caso transversal).
- **2. Generación:** registrar modelo, versión, configuración, *prompt*, herramientas.
- **3. Análisis preliminar:** revisión humana breve (formato, dependencias, supuestos).
- **4. SAST y SCA:** señales, no veredictos; triaje de hallazgos.
- **5. Compilación:** avisos como señales; puerta de entrada a la ejecución.
- **6. Sandbox:** sin red salvo excepción, cuotas, FS efímero, usuario no privilegiado.
- **7. Pruebas funcionales:** necesarias y no suficientes.
- **8. Property-based, fuzzing, sanitizers:** amplían la comprobación; ninguna cubre todas las rutas.
- **9. Evidencias:** recoger y asociar a cada requisito (sin trazabilidad no se audita).
- **10. Criterios de aceptación** y **11. revisión humana** (asume la responsabilidad).

Técnicas de análisis y sus límites

Técnica	Qué analiza	Cuándo	Falso positivo	Falso negativo
SAST	Código sin ejecutarlo	Antes de compilar	Patrón no alcanzable	Ruta no modelada; regla ausente
DAST	Aplicación en ejecución	Desplegado	Sonda no explotable	Cobertura insuficiente
SCA	Dependencias vs. catálogos	Al resolver dependencias	Vulnerable pero no alcanzable	No catalogada; no inventariada
SBOM	Inventario de componentes	En cada construcción	No es detección	Inventario incompleto
Fuzzing	Entradas generadas	En aislamiento	Señal espuria	Baja cobertura; oráculo
Análisis formal	Modelo y propiedades	Con especificación	No equiparable al triaje	Modelo incompleto
Sanitizers	Ejecución instrumentada	En el sandbox	Diagnóstico espurio	Ruta no ejercitada

SBOM no es un análisis

- Un SBOM es un *inventario* de componentes y versiones.
- Su valor es de trazabilidad y respuesta ante incidentes: saber qué se ve afectado.
- Confundir el inventario con la comprobación lleva a creer que «tener un SBOM» resuelve la cadena de suministro.

Ninguna técnica se presenta como «la que encuentra las vulnerabilidades»: el pipeline combina sesgos de error distintos; la combinación reduce la probabilidad de que un fallo pase inadvertido, pero no la anula.

Puertas de calidad del pipeline

Puerta	Criterio	Evidencia	Acción si no se cumple
Especificación	Requisitos declarados y revisables	Documento revisado	Rechazo
Estático	Sin hallazgos críticos confirmados abiertos	Informe y excepción autorizada	Bloqueo
Compilación	Compila; avisos tratados como señales	Registro de construcción	Rechazo
Aislamiento	Ejecución dentro del sandbox configurado	Registro de invocaciones	Rechazo de la ejecución
Funcional	Pruebas funcionales pasan	Resultados de pruebas	Rechazo
Dinámica	Sin fallos no explicados	Artefactos y trazas	Bloqueo
Evidencia	Trazabilidad completa	Matriz de trazabilidad	Bloqueo
Aceptación	Criterios cumplidos y revisión humana	Registro y aprobación	No se promueve

Fail-closed: ante duda o error, la puerta se cierra. Umbrales = **Propuesta docente**.

Orquestación conceptual fail-closed

```python
# PSEUDOCODIGO: sandbox.run debe implementarse y auditarse.
def require_ok(name, result):
    if result.returncode != 0 or not result.evidence_complete:
        raise RuntimeError(f"PUERTA FALLIDA: {name}")

require_ok("especificacion", check_specification())
require_ok("sast_sca", analyze_without_running_candidate())
require_ok("riesgos", check_findings_and_approved_exceptions())

policy = load_reviewed_sandbox_policy()
for name, command in [
    ("construccion", ["python", "build.py"]),
    ("funcionales", ["python", "-m", "pytest", "tests/funcionales"]),
    ("dinamicas", ["python", "run_dynamic.py"]),
]:
    require_ok(name, sandbox.run(command, policy=policy))

require_ok("evidencias", verify_traceability_and_artifacts())
```

Listing 1: Fail-closed: ninguna etapa que ejecute código candidato corre en el anfitrión.

Excepciones y casos de rechazo

- Una alerta se cierra por corrección verificada o por falso positivo demostrado.
- Un riesgo confirmado no corregido *no* queda resuelto con una explicación: requiere excepción limitada, documentada y autorizada, con plan de remediación.
- En docencia, bloquear siempre los hallazgos críticos confirmados.
- **Casos de rechazo típicos:** (i) especificación ausente; (ii) hallazgo crítico sin corrección; (iii) no compila o avisos silenciados; (iv) ejecución fuera del sandbox; (v) pruebas funcionales fallidas; (vi) fallo no explicado en dinámicas; (vii) trazabilidad incompleta; (viii) decisión humana no registrada.

Las puertas **no son garantías**: heredan los falsos negativos de las técnicas que las respaldan.

Integración de cambios concurrentes

- Validar un artefacto aislado no garantiza el sistema resultante al combinarlo con otros.
- **Conflicto textual:** el control de versiones lo detecta.
- **Conflicto semántico:** los cambios compilan y se combinan, pero juntos violan una propiedad que cada uno preservaba. Ninguna herramienta de fusión lo detecta por sí sola.
- La pregunta no es «¿pasa este cambio?», sino «¿pasa el estado que se va a integrar?».

La *cola de integración* reejecuta las comprobaciones sobre la rama principal actualizada: el estado validado es el que se integrará (**Evidencia** de ingeniería).

Del cambio aislado al sistema integrado

Nivel	Qué se valida	Límite
Cambio aislado	El artefacto cumple sus propiedades en su rama	No ve incompatibilidades con otros cambios
Cambio combinado	Conserva sus propiedades junto a los ya integrados	No cubre cambios no integrados
Estado de integración	El resultado sobre la rama actualizada	Reduce la ventana; no demuestra corrección del conjunto

Los conflictos semánticos que ninguna prueba ejerza seguirán siendo invisibles; las pruebas de integración heredan falsos negativos. **Propuesta docente**.

Trazabilidad: requisito, implementación, prueba y evidencia

Req.	Implementación	Prueba	Evidencia
R1	Lectura del CSV con validación canónica	Entradas malformadas; columnas ausentes	Registro y artefacto
R2	Rutas canónicas; sin seguir enlaces	Rutas relativas, symlinks, sustitución	Informe de rutas
R3	Temporales con funciones seguras	Colisiones y reutilización de nombres	Registro de temporales
R4	Límites de tamaño y tiempo	Entradas sobredimensionadas; lentitud	Registro de recursos
R5	Escapado de fórmulas	Prefijos de fórmula y variantes	Inspección del tipo de celda XLSX

La cadena *requisito* → *implementación* → *prueba* → *evidencia* es la forma concreta de la responsabilidad. Ejemplo de **Propuesta docente**.

Dependencias y entornos reproducibles

- Un resultado sobre una versión de dependencia distinta no es evidencia del artefacto a desplegar.
- **Fijar versiones** exactas (y huella de integridad), evitar referencias móviles.
- **Inventariar componentes:** SBOM generado en cada construcción y conservado.
- **Construir de forma declarativa:** entorno descrito en fichero versionado.
- **Fijar la imagen por contenido:** resumen criptográfico, no etiqueta móvil.
- **Registrar la procedencia** de cada dependencia y artefacto.

Reproducibilidad y aislamiento son propiedades *distintas y complementarias*: un entorno reproducible pero no aislado puede reproducir el daño; uno aislado pero no reproducible da resultados que no se repiten.

El límite del pipeline verde

- Superar todas las etapas significa que *no se ha observado ningún fallo bajo las comprobaciones ejecutadas*. No significa ausencia de vulnerabilidades.
- Es una batería *finita* sobre un conjunto *declarado* de propiedades; los falsos negativos se heredan.
- La especificación puede ser incompleta, las dependencias cambiar y el contexto diferir.

Un artefacto con pipeline verde es *validado contextualmente*, nunca «seguro» en sentido absoluto: superar una batería finita no demuestra seguridad.

Cómo presentar un resultado verde

1. *Qué* se comprobó: puertas y propiedades declaradas.
2. *Cómo* se comprobó: herramientas, versiones y entorno.
3. *Qué quedó fuera*: falsos negativos conocidos, requisitos no expresados, condiciones no probadas.

En lugar de «el artefacto es seguro», la formulación honesta es: «bajo las comprobaciones A, B y C, con las herramientas y versiones registradas, en el entorno descrito, no se observaron fallos; quedan sin comprobar las propiedades X e Y».

DISCUSIÓN «Si el pipeline está verde, el código es seguro»

- ¿Qué error conceptual comete esa afirmación y cómo reformularla correctamente?

Validación del propio pipeline

1. **Prueba fallida:** aserción falsa; comprobar código de salida y promoción bloqueada.
2. **Fallo de aislamiento:** deshabilitar una restricción; la política debe rechazar.
3. **Hallazgo confirmado:** ejemplo vulnerable; la puerta SAST/SCA lo bloquea si hay regla.
4. **Oráculo de seguridad:** XLSX con prefijos de fórmula; verificar tipo y contenido real.
5. **Evidencia ausente:** retirar un informe; la puerta de trazabilidad falla cerrada.

Si el pipeline termina en verde pese a esas perturbaciones, la propia infraestructura de validación requiere corrección. **Propuesta docente**, no certificación.

El kit AI-SDLC y su alcance

- El recurso `kit_agentes_ai_sdlc_v2` operacionaliza las puertas como **Propuesta docente**: no es estándar, no certifica, no sustituye la revisión humana.
- El runner distingue `PASS`, `FAIL`, `SKIP` y `NOT_EVALUATED`; un `SKIP` *no* es un `PASS`.
- Con `-strict` (o perfil `critical`) un `SKIP` bloquea la aceptación.
- `-sandbox` ejecuta en `bubblewrap`; si no está disponible, falla cerrado.

Que el runner termine en verde significa que las comprobaciones declaradas no fallaron en ese entorno, no que el artefacto sea seguro ni conforme a una norma.

Síntesis del capítulo

- *Validar* = comprobar propiedades *declaradas*; no es demostrar seguridad.
- El pipeline produce un *expediente* (qué, cómo, qué quedó fuera), no un veredicto binario.
- Las técnicas tienen sesgos de error distintos; combinarlas reduce, no anula, el riesgo de que un fallo pase inadvertido.
- Las puertas son *fail-closed* y no son garantías.
- La trazabilidad (requisito→implementación→prueba→evidencia) es la forma concreta de la responsabilidad.
- **Pipeline verde $\neq$ sin vulnerabilidades**: batería finita sobre propiedades declaradas.

Preguntas de reflexión

1. ¿Por qué el pipeline empieza en la especificación y no en el código?
2. Explicad falso positivo y falso negativo en SAST y en SCA; ¿por qué son estructurales?
3. ¿Por qué un SBOM no es una técnica de detección? ¿Para qué sirve en el expediente?
4. ¿Qué significa que una puerta sea *fail-closed*? Dad un ejemplo.
5. ¿Por qué el entorno reproducible y el aislamiento son propiedades distintas?

¿Qué propiedades quedan fuera de la automatización?

- ¿Qué propiedades de seguridad no son automatizables con estas técnicas y por qué quedan en la revisión humana?

Referencias I

Uso de IA generativa

Material elaborado con asistencia de IA generativa (DeepSeek-V4-Flash; considerados también GPT-6 y Claude Sonnet 4.6). Contenido revisado y validado por el autor.